



11.1. Operations on Trees

Previous Section:

 **11. Trees**

Contents:

1. Binary Trees

1.1. Special Types of Binary Trees

1.2. Representing Binary Trees

1.3. Maximum Number of Nodes in a Binary Tree

1.4. Limitation of Array Representation

1.5. Linked Representation of Binary Trees

1.6. Advantages of Linked Representation

2. Tree Traversal

2.1. Breadth-First Traversal

2.2. Depth-First Traversal

Summary

References

So far, we have learned how trees organize data into a hierarchical structure. However, a tree becomes truly useful only when we can manipulate it efficiently.

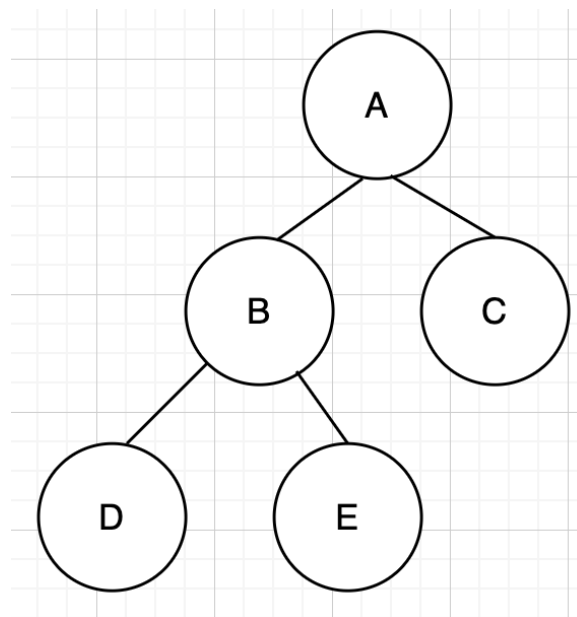
In this section, we will introduce the most common operations performed on trees. These operations allow us to create, traverse, search, insert, delete, and modify nodes within a tree structure. Each operation serves a specific purpose and helps us manage hierarchical data effectively.

Understanding these operations is important because many real-world applications rely on trees, from file systems and databases to search engines and artificial intelligence. By mastering these fundamental operations, we can better understand how tree-based data structures store, access, and organize information.

1. Binary Trees

A **Binary Tree** is a tree in which each node can have **at most two child nodes**. In other words, every node may have no child node, one child node, two child nodes

The two children are commonly referred to as the **left child** and **right child**.



In the example above:

- A has two children: B and C
- B has two children: D and E
- C, D, and E have no children

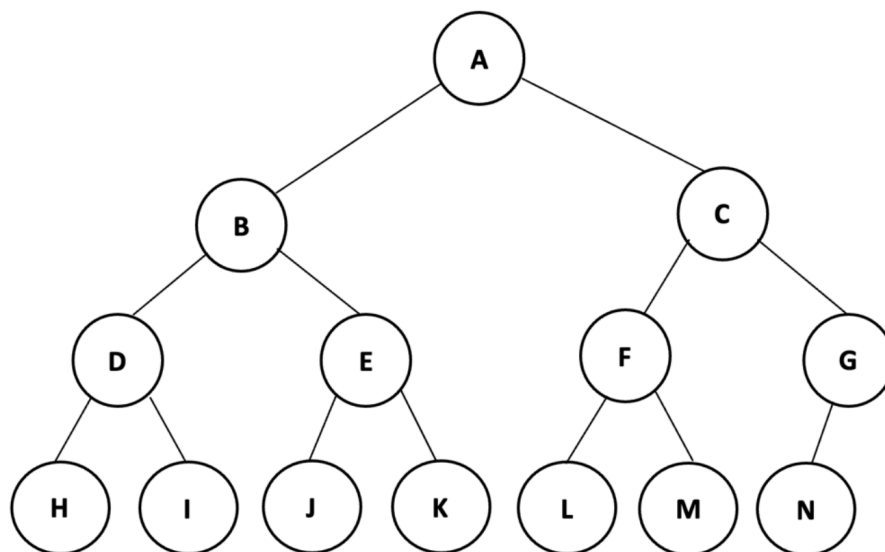
Because no node has more than two children, this is a Binary Tree. Similarly, a tree where each node can have up to **three children** is called a **Ternary Tree**.

More generally, if each node can have up to **N children**, the structure is called an **N-ary Tree**.

1.1. Special Types of Binary Trees

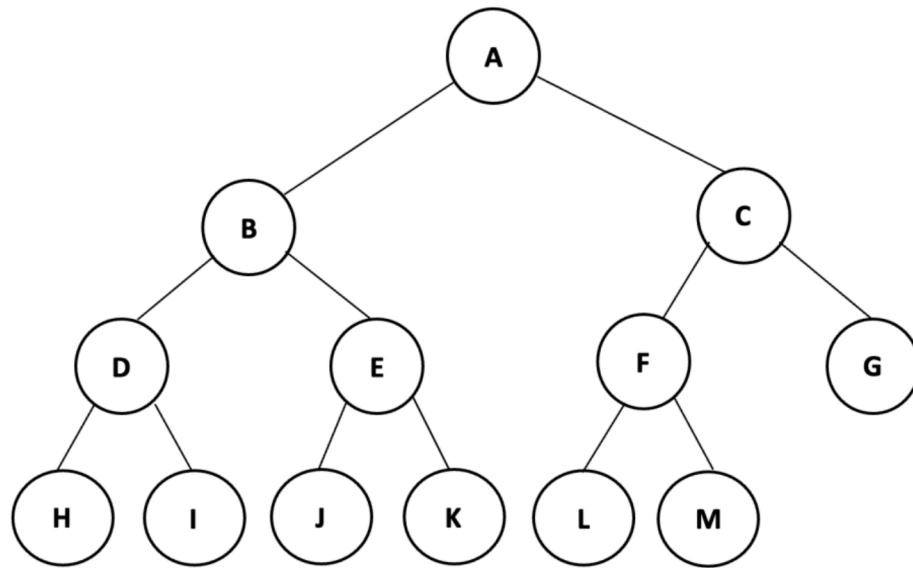
Binary Trees can be classified into several categories based on how their nodes are arranged.

- Complete Binary Tree - A **Complete Binary Tree** is a binary tree in which:
 - Every level, except possibly the last level, is completely filled.
 - The last level is filled from left to right.



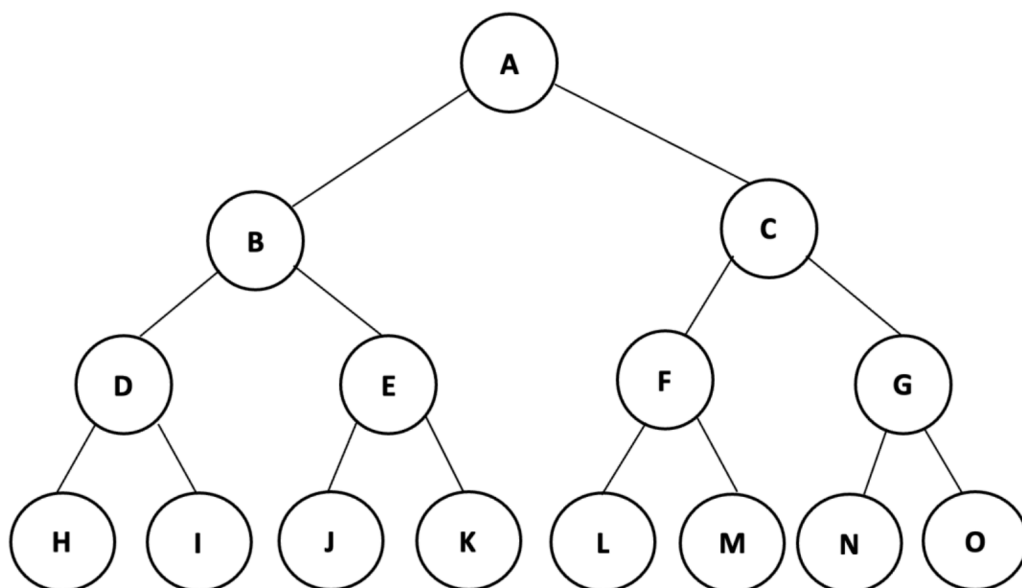
This is a Complete Binary Tree because all levels are filled except the last, and the last level fills from the left side first.

- **Full Binary Tree:** A **Full Binary Tree** is a binary tree in which every non-leaf node has exactly two children.



This is a Full Binary Tree because every internal node has exactly two children.

- **Perfect Binary Tree** - A **Perfect Binary Tree** satisfies two conditions:
 - Every internal node has exactly two children.
 - All leaf nodes are located at the same level.



All leaves (D, E, F, and G) are on the same level, making this a Perfect Binary Tree.



Important property: A Perfect Binary Tree is always Full and Complete.

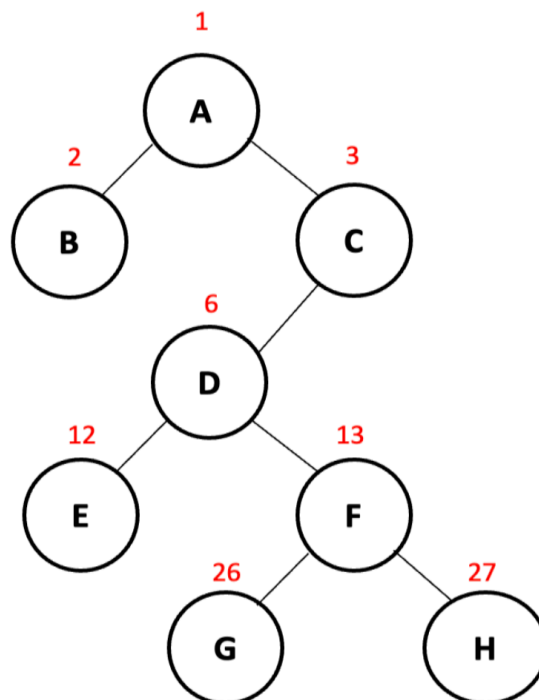
1.2. Representing Binary Trees

Binary Trees can be represented in two common ways: Using an array or linked list.

Array Representation of Binary Trees: One method of storing a Binary Tree is to place its nodes inside an array. To do this, each node is assigned a position based on its relationship with its parent.

- Assume a node occupies position k . Then:
 - **Left Child** = $2k$
 - **Right Child** = $2k + 1$

Consider the following tree:



The number inside each bracket represents the node's position according to the array formula.

- For example:

- $A = 1$

Left Child of $A = 2 \times 1 = 2$ (B)

Right Child of $A = 2 \times 1 + 1 = 3$ (C)

- $D = 6$

Left Child of $D = 2 \times 6 = 12$ (E)

Right Child of $D = 2 \times 6 + 1 = 13$ (F)

- **Corresponding Array** - The tree can be stored as:

A	B	C			D						E	F		G	H
0	1	2	3	4	5	6	7	8	9	10	11	12	...	25	26

Where: "-" indicates an empty position. Missing child nodes leave unused spaces in the array. The position of each node follows the Binary Tree indexing rule.

Notice that several positions remain empty because the tree is not complete.

1.3. Maximum Number of Nodes in a Binary Tree

For a Binary Tree with depth n , the maximum number of nodes occurs when the tree is a Perfect Binary Tree. The total number of nodes is:

$$2^0 + 2^1 + 2^2 + \dots + 2^n$$

This geometric series simplifies to: $2^{n+1} - 1$

Suppose a Binary Tree has depth 4. The maximum number of nodes is:

$$2^0 + 2^1 + 2^2 + 2^3 + 2^4$$

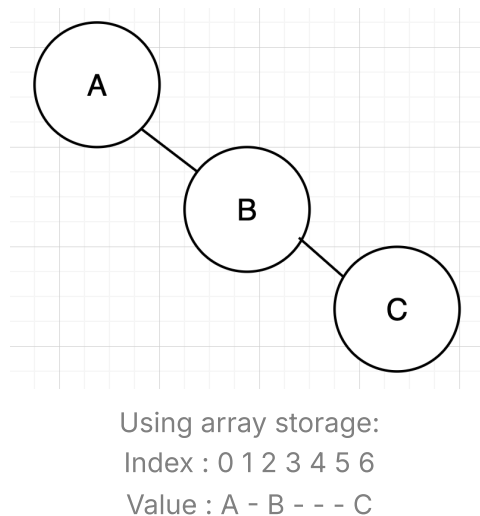
Which becomes: $1 + 2 + 4 + 8 + 16 = 31$

Or directly: $2^{4+1} - 1 = 32 - 1 = 31$

Therefore, an array storing a depth-4 Binary Tree must have space for 31 nodes, with indices: $0 \rightarrow 30$

1.4. Limitation of Array Representation

Array representation works efficiently when the tree is complete or nearly complete. However, if the tree contains many missing nodes, large portions of the array remain unused.



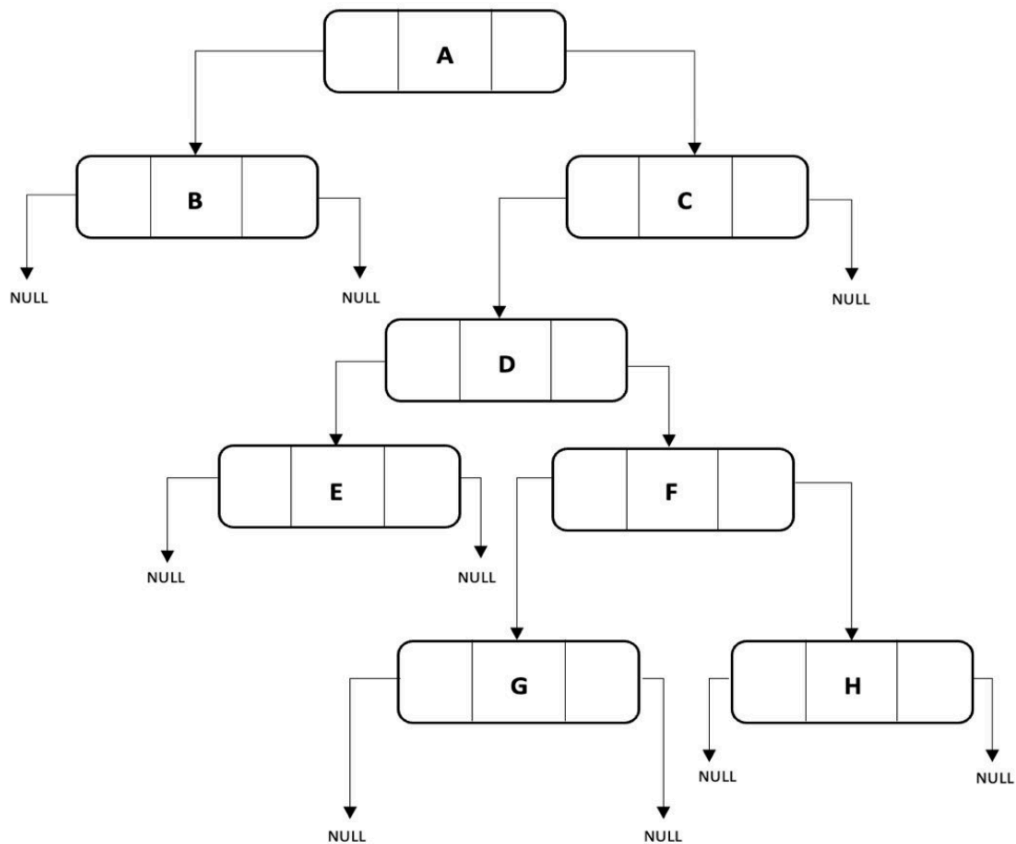
Most positions are empty. As the tree becomes larger, the amount of wasted memory increases significantly.

1.5. Linked Representation of Binary Trees

To overcome the memory waste problem, Binary Trees are commonly stored using linked structures. Each node contains three components:

- Data: Stores the value of the node → `Data = A`
- Left Pointer: Stores a reference to the node's left child.
- Right Pointer: Stores a reference to the node's right child.

Example: The same tree above can be visualized as:



When a node has no child, the corresponding pointer stores: NULL

1.6. Advantages of Linked Representation

Linked storage provides several benefits:

- No wasted memory from unused array positions.
- Nodes can be inserted dynamically.
- Nodes can be removed dynamically.
- The structure can grow and shrink without reorganizing existing nodes.
- Suitable for sparse and unbalanced trees.

Because of these advantages, linked representation is the most commonly used method for implementing Binary Trees in practical applications.

2. Tree Traversal

One of the most common operations performed on a tree is **traversal**. Traversal simply means visiting every node in the tree according to a specific order.

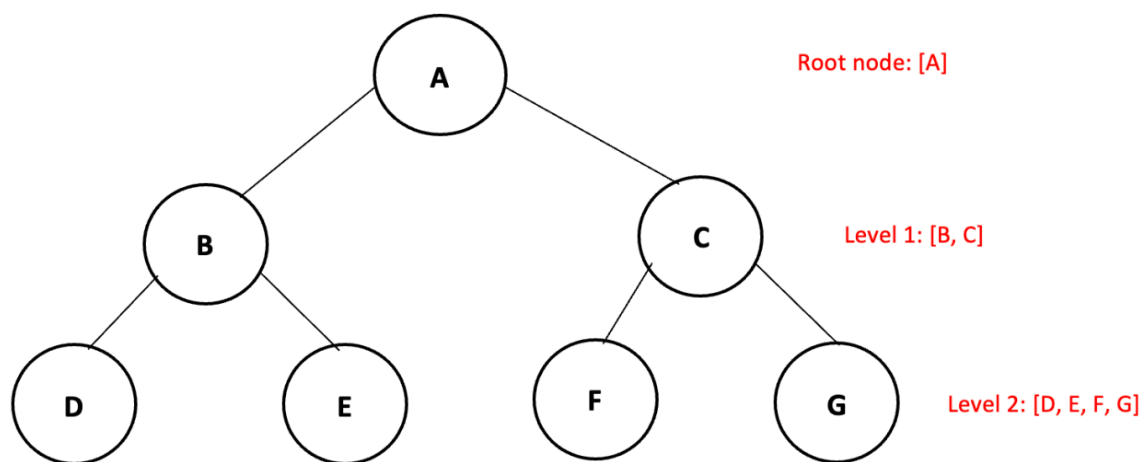
When working with trees, there are two major categories of traversal: **Breadth-First Traversal (BFS)** and **Depth-First Traversal (DFS)**. Although both methods eventually visit every node, the order in which they visit the nodes is completely different.

2.1. Breadth-First Traversal

Let's start with the more intuitive one.

In **Breadth-First Traversal**, we visit nodes level by level. We begin at the root node, then visit all nodes on the next level, followed by all nodes on the level below that, and continue until every node has been visited. Think of it as reading a tree from top to bottom.

For the tree below:



The traversal order would be: [A, B, C, D, E, F, G]

We first visit the root (**A**), then all of its children (**B** and **C**), and finally the children of those nodes (**D**, **E**, **F**, and **G**).

Since every node is visited exactly once, the time complexity of Breadth-First Traversal is: $O(n)$, where n is the total number of nodes in the tree.

2.2. Depth-First Traversal

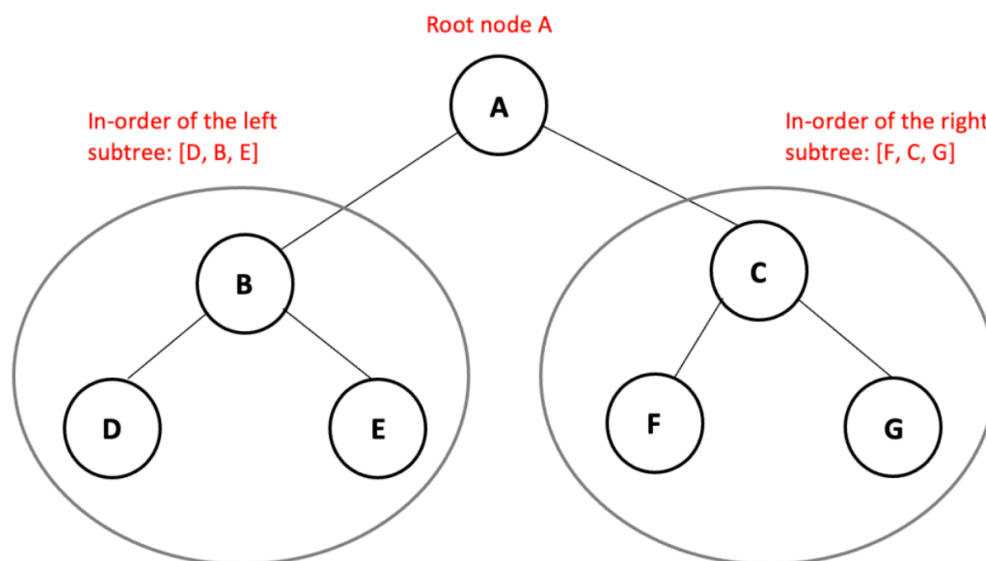
While Breadth-First Traversal explores the tree level by level, **Depth-First Traversal** follows a different philosophy.

Instead of exploring an entire level first, we continue moving deeper into a branch until no further progress can be made, then backtrack and explore the remaining branches.

For a binary tree, there are three common Depth-First Traversal methods: In-order Traversal; Post-order Traversal; and Pre-order Traversal

- **In-order Traversal:** In an **In-order Traversal**, we visit **Left Subtree** → **Root Node** → **Right Subtree**
 - The order can be summarized as: Left → Root → Right

Using the same tree:



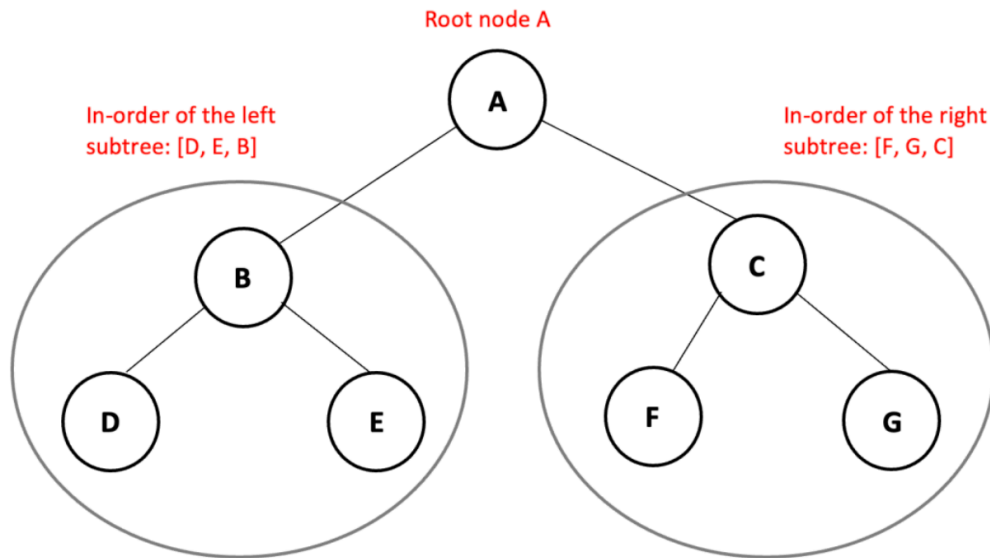
The traversal order becomes: [D, B, E, A, F, C, G]

One reason In-order Traversal is important is that, in a **Binary Search Tree (BST)**, it visits the nodes in ascending order.

- **Post-order Traversal:** In a **Post-order Traversal**, we visit **Left Subtree** → **Right Subtree** → **Root Node**
 - The order can be summarized as: Left → Right → Root.

- Notice that the root node is always visited last.

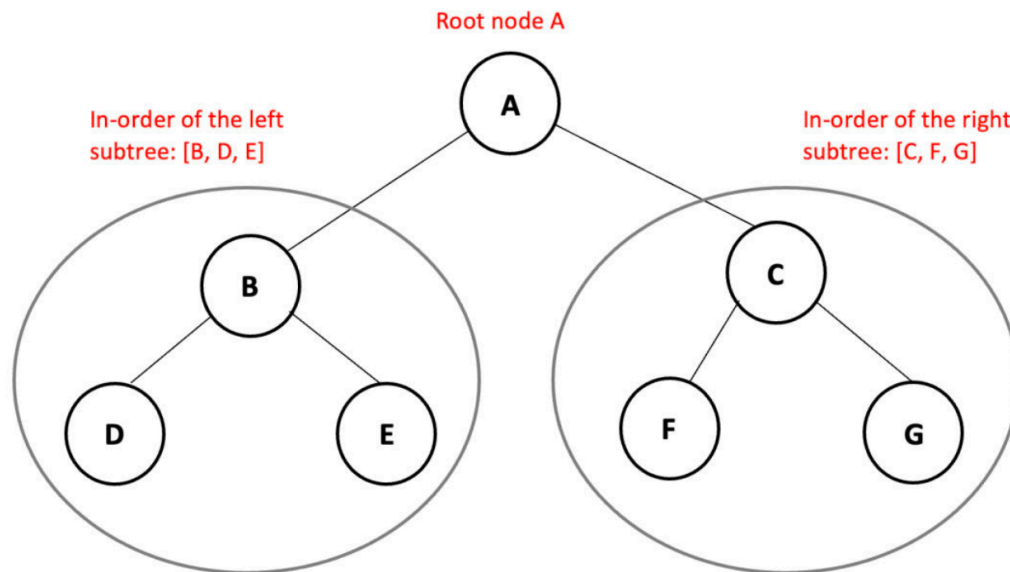
For our example tree: [D, E, B, F, G, C, A]



Post-order Traversal is particularly useful when deleting or freeing a tree because child nodes are processed before their parent node.

- **Pre-order Traversal:** In a **Pre-order Traversal**, we visit Root Node → Left Subtree → Right Subtree
 - The order can be summarized as: Root → Left → Right
 - The root node is therefore always visited first.

For our example tree: [A, B, D, E, C, F, G]



Pre-order Traversal is commonly used when copying a tree or generating a hierarchical representation of the structure.

- **Time Complexity of Depth-First Traversals**

Whether we use In-order Traversal; Post-order Traversal; Pre-order Traversal, every node is still visited exactly once. Therefore, the time complexity for all Depth-First Traversal methods is $O(n)$

These traversal algorithms are commonly implemented using **recursion**, where each subtree is treated as a smaller version of the original tree.

Summary

In this section, we explored the fundamentals of **Trees**, one of the most important hierarchical data structures in computer science. We learned the structure of a **Binary Tree**, including its common types such as full, complete, perfect, and balanced binary trees.

We then examined two common methods for representing trees in memory: **array-based representation** and **linked-list representation**, each with its own advantages depending on the application.

Finally, we studied **tree traversal**, the process of visiting every node in a tree. This included **Breadth-First Traversal (BFS)** and the three Depth-First Traversal (DFS) methods: **Pre-order**, **In-order**, and **Post-order** traversal.

Understanding these concepts provides the foundation for more advanced tree structures, particularly the **Binary Search Tree (BST)**, which we will explore in the next section.

References

https://drive.google.com/file/d/1tOqiD1hwN4q_ZbZHeOp-e3oeJfgTEdDq/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/types-of-binary-tree/>

<https://www.geeksforgeeks.org/dsa/tree-traversals-inorder-preorder-and-postorder/>

<https://builtin.com/software-engineering-perspectives/tree-traversal>

Next Section:

 [11.2. Binary Search Trees](#)